

Numerical Optimization for Large Scale Problems

Lecture Notes

Francesco Della Santa *

Politecnico di Torino, A.Y. 2025/2026

Abstract

Notes of the optimization laboratories held by Francesco Della Santa for the course of Numerical Optimization for Large Scale Problems.

Contents

1	Direct Methods for Linear Systems	4
1.1	Square Matrices	4
1.2	Gaussian Elimination	5
1.3	Square Linear Systems	5
1.4	Condition Number	6
1.5	Direct Methods in Matlab	6
1.6	Direct Methods in Python	6
1.7	Fill-In Phenomenon	9
1.8	Exercises	9
2	Finite Differences	11
2.1	Finite Differences (Gradient)	11
2.1.1	Choice of h	11
2.2	Finite Differences (Hessian)	12
2.2.1	Choice of h	12
2.3	Finite Differences (Jacobian)	12
2.4	Exercises	12
3	Iterative Methods for Linear Systems	14
3.1	The Gradient Method	15
3.1.1	Sequence of the Gradient Method	16
3.1.2	Convergence Properties	17
3.2	Conjugate Gradient Method	17
3.2.1	Preconditioning	18
3.3	Other Iterative Methods	19
3.4	Exercises	19
4	Steepest Descent	21
4.1	Exercises	21

*Dipartimento di Scienze Matematiche, Politecnico di Torino, Turin, Italy

5	Steepest Descent with Backtracking	23
5.1	Backtracking	23
5.2	Exercises	24
A	Material for Refreshing Basic Knowledge	26
B	Test Functions	26
B.1	The Rosenbrock and the Himmelblau Test Functions	26
B.2	n -dimensional Test Function	27

Introduction

In this document, we report the text of the laboratories for the course of *Numerical Optimization for Large Scale Problems*, A.Y. 2025/2026. Each section corresponds to one laboratory and it will consist of a brief theoretical recap of the arguments, followed by the exercises.

Usually (but not always), the solutions of the exercises of each laboratory are published on the course's web page, before the start of the next one. Solutions will be published only as MATLAB code (except for the first laboratory, published also as Python code).

Remark 0.1 (Material for refreshing knowledge: MATLAB and basic numerical methods for Linear Algebra). In Section [A](#), we report the content of the message of Prof. Pieraccini on the course web page and related to the available material about “*MATLAB pills*” and “*Background*”. Look at that material to refresh your knowledge about MATLAB and basic numerical methods for Linear Algebra, since they are strongly suggested for the laboratories (and the course in general).

1 Direct Methods for Linear Systems

Let us consider a linear system, with coefficients in $\mathbb{R}$, of m equations and n unknowns

$$A\mathbf{x} = \mathbf{b}, \quad (1)$$

where $A \in \mathbb{R}^{m \times n}$ denotes the matrix of the *coefficients*, $\mathbf{x} \in \mathbb{R}^n$ denotes the vectors of the *unknowns*, and $\mathbf{b} \in \mathbb{R}^m$ is the vector of *known terms*.

Then, we recall that:

1. **Rouché-Capelli's Theorem:** system (1) admits solutions if and only if

$$\text{rank}(A) = \text{rank}(A|\mathbf{b}). \quad (2)$$

In particular, if (2) holds, (1) admits $\infty^{n-\text{rank}(A)}$ solutions (one solution if $\text{rank}(A) = n$).

2. If $\mathbf{b} = \mathbf{0}$, the system (1) is defined as *homogeneous system*. Due to item 1, a homogeneous system always admits solutions (trivial solution: $\mathbf{x}^* = \mathbf{0}$).
3. If $\mathbf{b} \neq \mathbf{0}$ The set of all and only the solutions of system (1) is

$$\mathcal{S}_{\mathbf{b}} = \{\bar{\mathbf{x}} + \mathbf{x}^* \mid \mathbf{x}^* \in \mathcal{S}_{\mathbf{0}}\}, \quad (3)$$

where $\bar{\mathbf{x}} \in \mathbb{R}^n$ is a particular solution of (1) (i.e., $A\bar{\mathbf{x}} = \mathbf{b}$) and $\mathcal{S}_{\mathbf{0}} \subseteq \mathbb{R}^n$ is the set of all and only the solutions of the homogeneous system $A\mathbf{x} = \mathbf{0}$.

1.1 Square Matrices

Here, we focus on the case of linear systems like (1) but where the number of equations is equal to the number of unknowns. Therefore, the matrix of the coefficients is a square matrix $A \in \mathbb{R}^{n \times n}$ and the vector of known terms is $\mathbf{b} \in \mathbb{R}^n$ (i.e., $m = n$).

We recall the following types of square matrices:

- **Diagonal:** A is diagonal if $a_{ij} = 0$, for each $i \neq j$.

$$A = \begin{bmatrix} \bullet & & & \\ & \ddots & & \\ & & \ddots & \\ & & & \bullet \end{bmatrix}.$$

- **Lower/Upper Triangular:** A is lower/upper triangular if $a_{ij} = 0$, for each $i \gtrless j$.

$$A_{lt} = \begin{bmatrix} \bullet & & \\ \vdots & \ddots & \\ \bullet & \dots & \bullet \end{bmatrix}, \quad A_{ut} = \begin{bmatrix} \bullet & \dots & \bullet \\ & \ddots & \vdots \\ & & \bullet \end{bmatrix}.$$

- **Symmetric:** A is symmetric if it is equal to its transpose (i.e., $A = A^T$).
- **Positive/Negative (Semi-)Definite:** A is positive/negative definite if $\mathbf{x}^T A \mathbf{x} \gtrless 0$, for each $\mathbf{x} \in \mathbb{R}^n \setminus \{\mathbf{0}\}$. A is positive/negative semi-definite if $\mathbf{x}^T A \mathbf{x} \gtrless= 0$, for each $\mathbf{x} \in \mathbb{R}^n \setminus \{\mathbf{0}\}$.
- **Invertible and Inverse Matrix:** A is *invertible* if exists a matrix $B \in \mathbb{R}^{n \times n}$ such that $AB = BA = \mathbb{I}_n$. Then, B is defined as *inverse matrix* of A and is denoted by A^{-1} .
A matrix that is not invertible is also called *singular*; then, saying that a matrix is *non-singular* is equivalent to saying that it is *invertible*.
- **Orthogonal:** A is *orthogonal* if its inverse is equal to its transpose (i.e., $A^{-1} = A^T$ and $AA^T = A^T A = \mathbb{I}_n$).

1.2 Gaussian Elimination

Here, we briefly recall the *Gaussian Elimination* properties for matrices and linear systems. First of all, we recall that the gaussian elimination operations are:

- **Switch:** switch row i and row j of a matrix;
- **Multiplication by Scalar:** multiply the i -th row of a matrix by a scalar value $\lambda \in \mathbb{R}$;
- **Add Row to Another:** add row i , multiplied by a scalar $\lambda \in \mathbb{R}$, to row j of a matrix.

We recall that the gaussian elimination operations have the following properties:

- They are invertible linear applications; i.e., their action can be described by left-multiplying A by a non-singular matrix $E \in \mathbb{R}^{m \times m}$.
- Let $E_1, \dots, E_k$ be the ordered sequence of $k \in \mathbb{N}$ gaussian elimination operations applied to A and let $E \in \mathbb{R}^{m \times m}$ be $E := E_k \cdots E_1$. Then:
 - E is non-singular;
 - The linear system $A'x = b'$, with $A' = EA$ and $b' = Eb$, has the same solutions of (1).

1.3 Square Linear Systems

Concerning square linear systems, we recall that:

1. **A invertible (i.e., non-singular):** due to Rouché-Capelli's Theorem (see item 1 above), if $\det(A) \neq 0$, then (1) admits one unique solution. Moreover, since A is invertible if and only if $\det(A) \neq 0$, the unique solution of (1) is $x^* = A^{-1}b$.
2. **A not invertible (i.e., singular):** if A is singular, the solutions of the linear system depends on the Rouché-Capelli's Theorem.
3. **Forward/Backward Substitution:** if A is lower/upper triangular and non-singular, the unique solution x^* can be easily computed using *forward/backward substitution*. We recall that all the diagonal elements of a non-singular triangular matrix are non-null (i.e., $a_{ii} \neq 0$, for each $i = 1, \dots, n$).

- **Forward Substitution:**

$$x_1^* = \frac{b_1}{a_{11}} \rightarrow x_2^* = \frac{b_2 - a_{21}x_1^*}{a_{22}} \rightarrow \dots \rightarrow x_n^* = \frac{b_n - a_{n(n-1)}x_{(n-1)}^* - \dots - a_{n1}x_1^*}{a_{nn}}.$$

- **Backward Substitution:**

$$x_n^* = \frac{b_n}{a_{nn}} \rightarrow x_{(n-1)}^* = \frac{b_{(n-1)} - a_{(n-1)n}x_n^*}{a_{(n-1)(n-1)}} \rightarrow \dots \rightarrow x_1^* = \frac{b_1 - a_{12}x_2^* - \dots - a_{1n}x_n^*}{a_{11}}.$$

4. **Substitution (Diagonal Special Case):** if A is non-singular and diagonal, the unique solution of (1) is $x^* = (b_1/a_{11}, \dots, b_n/a_{nn})$.
5. **Gaussian Elimination:** if A is non-singular we can perform a gaussian elimination $E \in \mathbb{R}^{n \times n}$ such that EA is a triangular matrix (usually, upper triangular); then, we solve the triangular linear system $(EA)x = Eb$ by substitution.
6. **LU Factorization:** let A be non-singular and E the matrix of gaussian elimination such that $EA = U$ is upper triangular and E^{-1} is lower triangular; then, $LU = E^{-1}EA = A$ and we can solve the linear system (1) by two substitutions: $y^* = L^{-1}b$, $x^* = U^{-1}y^*$.

7. **Cholesky Factorization:** if A is symmetric and positive definite, there is a unique upper triangular matrix R , with positive diagonal elements, such that the LU factorization of A is $A = R^T R$.
8. **QR Factorization:** with proper operations (other than gaussian elimination), A can be decomposed as $A = QR$, where Q is orthogonal and R is upper triangular.

Observation: factorizations typically are more convenient than the gaussian elimination if you have to solve many linear systems with the same matrix (e.g., $A\mathbf{x} = \mathbf{b}_1, \dots, A\mathbf{x} = \mathbf{b}_N$).

Observation: the advantages of the QR factorization are:

- methods for obtaining the QR factorization are usually *more stable* (see Section 1.4 below), but more expensive, than the ones used for obtaining the LU factorization;
- QR factorization can be easily extended to *rectangular* linear systems.

1.4 Condition Number

The reliability of a solution $\mathbf{x}^*$ of system (1) depends on the condition number of the matrix A . We recall that the condition number of A is

$$k(A) = \|A\| \|A^{-1}\|, \quad (4)$$

where $\|\cdot\|$ denotes a matrix norm (typically, it is used $\|\cdot\|_2$).

Specifically, let $A \in \mathbb{R}^{n \times n}$ be a non-singular matrix and let $\mathbf{x}, \mathbf{b} \in \mathbb{R}^n$ be two vectors such that $A\mathbf{x} = \mathbf{b}$. Moreover, let $\hat{\mathbf{x}} := \mathbf{x} + \delta\mathbf{x}$ and $\hat{\mathbf{b}} := \mathbf{b} + \delta\mathbf{b}$, with $\delta\mathbf{x} \neq \mathbf{0}$ and $\delta\mathbf{b}$ such that $A\hat{\mathbf{x}} = \hat{\mathbf{b}}$.

Then

$$\frac{\|\delta\mathbf{x}\|}{\|\mathbf{x}\|} \leq k(A) \frac{\|\delta\mathbf{b}\|}{\|\mathbf{b}\|}, \quad (5)$$

where here $\|\cdot\|$ denotes a vector norm (typically, it is used $\|\cdot\|_2$).

Observation: $k(A) \geq \|AA^{-1}\| = \|\mathbb{I}_n\| = 1$; i.e., $k(A) \in [1, +\infty)$.

Observation: the smaller $k(A)$, the more reliable the solution $\mathbf{x}^*$ of system (1). On the contrary, the larger $k(A)$, the larger the upper bound for the noise of the solution and, by consequence, the lower the reliability of $\mathbf{x}^*$.

If a matrix A is characterized by a large condition number (i.e., $k(A) \gg 1$), the matrix is *ill-conditioned*.

1.5 Direct Methods in Matlab

In Figures 1 and 2, the schemes of the Matlab procedure for solving linear systems are reported. In particular, we have two different schemes depending on the method used to store matrices: *dense* matrices and *sparse* matrices. In general, for solving a linear system in Matlab using a direct method, the `mldivide` function is used (equivalent to the operator `\`). [Here](#), the online documentation.

1.6 Direct Methods in Python

For solving linear systems in Python, the `numpy` and `scipy` modules are strongly suggested; indeed, they have many built-in linear algebra tools. In general, the sub-modules `numpy.linalg` and `scipy.linalg` are almost equivalent; on the other hand, for working with sparse matrices, the sub-modules `scipy.sparse` and `scipy.sparse.linalg` are necessary. In particular, for solving linear systems you can use `numpy.linalg.solve` or `scipy.linalg.solve` for dense matrices and `scipy.sparse.linalg.spsolve` for sparse matrices. The schemes adopted by these procedures are similar to the ones of `mldivide` in Matlab.

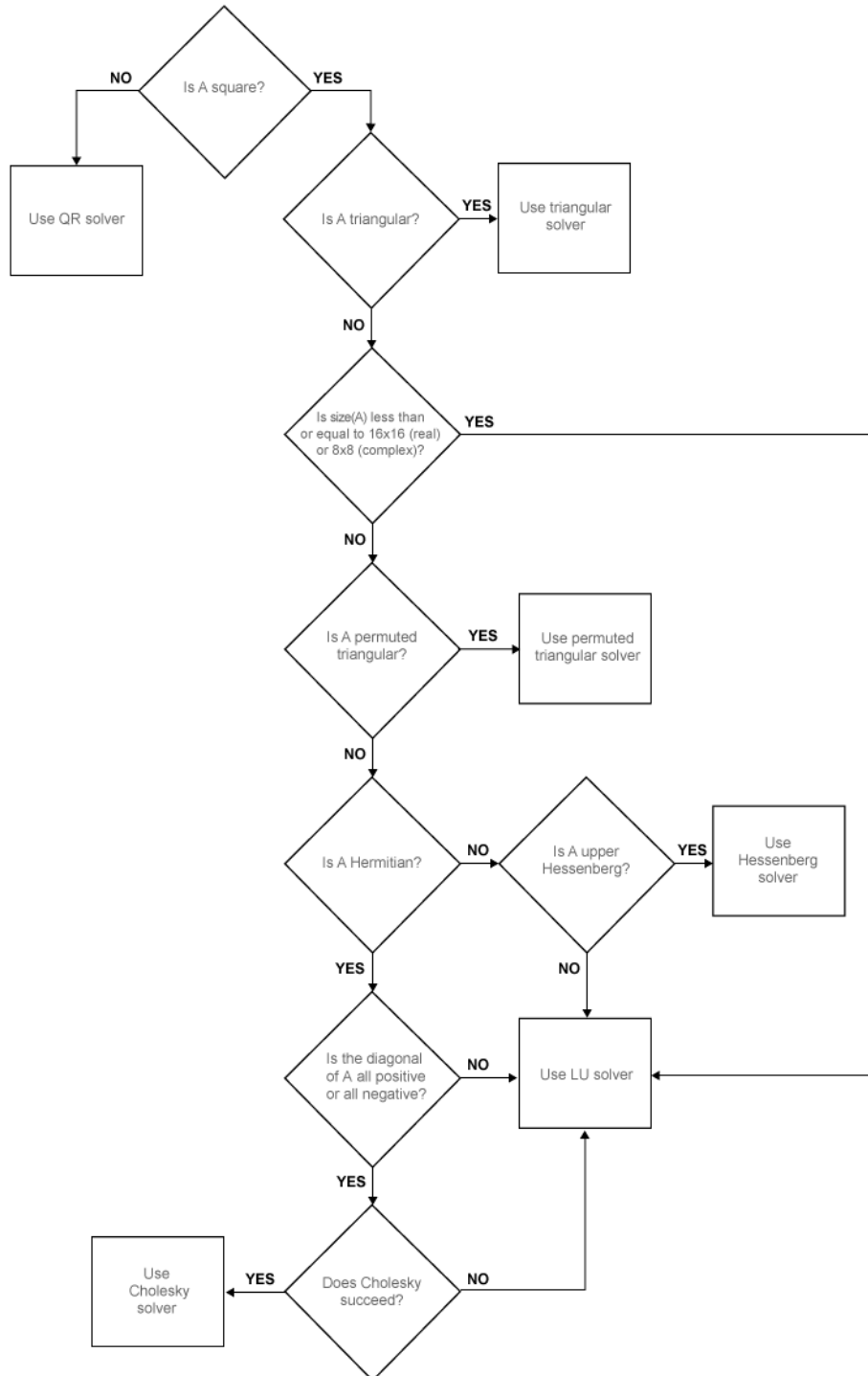


Figure 1: Scheme for the `mldivide` (`\`) function in Matlab (dense matrices case).

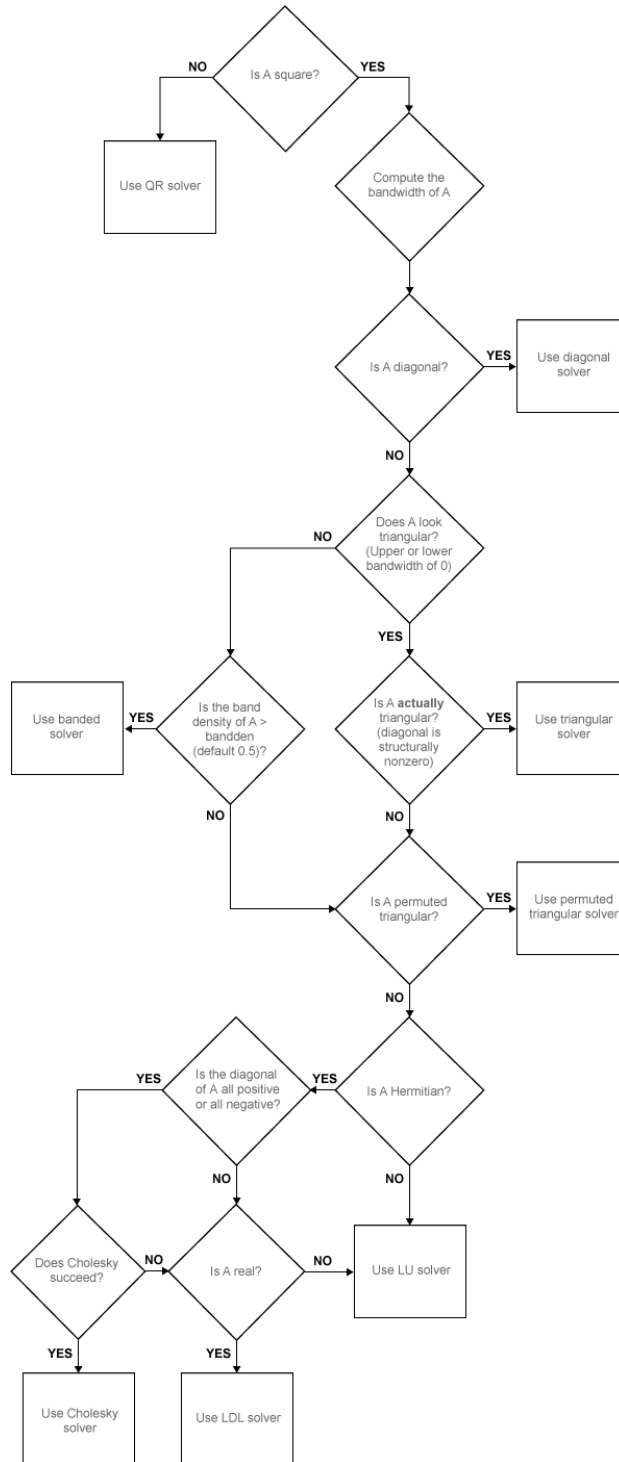


Figure 2: Scheme for the `mldivide` (`\`) function in Matlab (sparse matrices case).

1.7 Fill-In Phenomenon

We recall that a matrix is *sparse* if most of its elements are equal to zero. Sparse matrices have specific storage methods for reducing memory storage without losing efficiency in matrix operations.

Nonetheless, direct methods are not suitable for very large sparse matrices, due to the *fill-in* phenomenon. In particular, we have that a factorization of a sparse matrix A can return two *dense* matrices. This phenomenon can lead to memory storage problems and/or expensive computations of the factors.

1.8 Exercises

Exercise 1.1 (Basic Linear Systems). Write a script file where:

1. The random seed is 1.
2. A matrix $A \in \mathbb{R}^{n \times n}$ is initialized with elements randomly sampled with uniform distribution in $[0, 1]$. The value of n can be initialized as $n = 5$ for simplicity.
3. A vector $\mathbf{b} \in \mathbb{R}^n$ is initialized such that the exact solution of $A\mathbf{x} = \mathbf{b}$ is the vector $\mathbf{x}^* = \mathbf{e} = (1, \dots, 1) \in \mathbb{R}^n$.
4. A set of N vectors $\mathbf{b}_1, \dots, \mathbf{b}_N \in \mathbb{R}^n$ are initialized such that $\mathbf{b}_i = \mathbf{b} + \boldsymbol{\xi}_i$, $\boldsymbol{\xi}_i \in \mathcal{U}([0, 1]^n)$, for each $i = 1, \dots, N$. The value of N can be initialized as $N = 10$ for simplicity.
5. A diagonal matrix $D \in \mathbb{R}^{n \times n}$ is initialized with elements randomly sampled with uniform distribution in $[0, 1]$.
6. Solve the linear systems $A\mathbf{x} = \mathbf{b}$, $D\mathbf{x} = \mathbf{b}$, and $A\mathbf{x} = \mathbf{b}_i$, for each $i = 1, \dots, N$.
7. Compute the norms of the residuals of the numerical solutions; i.e., $\|A\hat{\mathbf{x}} - \mathbf{b}\|$, $\|D\hat{\mathbf{x}} - \mathbf{b}\|$, and $\|A\hat{\mathbf{x}} - \mathbf{b}_i\|$, for each $i = 1, \dots, N$.
8. Compute the condition number of A and D and comment the results.

Exercise 1.2 (Ill-Conditioned Problem). Write a script file where you solve a linear system $A\mathbf{x} = \mathbf{b}$ such that

$$A = \begin{bmatrix} 0.835 & 0.667 \\ 0.333 & 0.266 \end{bmatrix},$$

and $\mathbf{b} \in \mathbb{R}^2$ is the result of an experiment, measured using an instrument with tolerance ± 0.001 . For simplicity, assume to have the following 11 measurements of $\mathbf{b}$:

$$\mathbf{b}_i = \mathbf{b}^* - 0.001 + (i - 1)0.0002, \quad \forall i = 1, \dots, 11,$$

where $\mathbf{b}^* = (0.168, 0.067)$ is the exact vector of known terms.

Then:

1. Solve the linear systems $A\mathbf{x} = \mathbf{b}_i$, for each $i = 1, \dots, N$.
2. Compute the norms of the residuals of the numerical solutions; i.e., $\|A\hat{\mathbf{x}} - \mathbf{b}_i\|$, for each $i = 1, \dots, N$.
3. Compute the condition number of A and comment the results.

Exercise 1.3 (Problem Conditioning). Write a function that takes as inputs a random seed, the dimension n and the number of samples N , and that runs the same operations of the script of Exercise 1.1, returning:

1. $A, \mathbf{b}$, and D ;
2. the solutions of $A\mathbf{x} = \mathbf{b}$, $D\mathbf{x} = \mathbf{b}$, and $A\mathbf{x} = \mathbf{b}_i$, for each $i = 1, \dots, N$;
3. $k(A)$ and $k(D)$.

Try the function for different random seeds (e.g., 42) and try to understand if the linear systems' solutions are reliable or not. Then, try also to change dimension n and number of samples N .

Exercise 1.4. Download, run, and read the script `lab01_sparse_examples.m/.py` carefully. Then write a new script file where, for each $n = 200, 400, \dots, 2000$, the following operations are performed:

- Initialize a random sparse matrix $A \in \mathbb{R}^{n \times n}$ with density 0.01 (0.1 in Python);
- Compute the exact density of non-zero elements of A ;
- Compute $L, U \in \mathbb{R}^{n \times n}$ through the LU Factorization of A ;
- Compute the exact density of non-zero elements of L and U ;
- Update the script file, increasing the maximum value of n and looking for the value that run out of memory your PC.

In the end, plot the evolution of the density values for A , L , and U , with respect to n and comment the results. Moreover, spy the collocation of the non-zero values of the last matrices A , L , and U computed.

Suggestion: write a function that takes a random seed, the dimension n , and the desired density of A as inputs, and that returns all you need to perform the operations asked by the exercise.

Exercise 1.5. Write a new script that performs the same operations of Exercise 1.4 but using only dense matrices. Observe when your PC runs out of memory.

Exercise 1.6. Write a new script that performs the same operations of Exercise 1.4 but using the incomplete LU factorization of the matrix $\tilde{A} = A + \mathbb{I}$. Measure and spy the density of the factorization matrices $\tilde{L}, \tilde{U}$, and the compute error norm $\| \tilde{L}\tilde{U} - \tilde{A}P \|$ and the relative error norm $\| \tilde{L}\tilde{U} - \tilde{A}P \| / \| A \|$.

2 Finite Differences

We briefly recall the main Finite Difference formulas for approximating derivatives, to help carry out the exercises of this section.

Notation 2.1 (Notations for Gradient, Hessian, and Jacobian). *Let $f : \Omega \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$, $\mathbf{F} = (f_1, \dots, f_m) : \Omega \subseteq \mathbb{R}^n \rightarrow \mathbb{R}^m$, then:*

- Gradient of f (differentiable in Ω):

$$\nabla f = \begin{bmatrix} f_{x_1} \\ \vdots \\ f_{x_n} \end{bmatrix} : \mathbb{R}^n \rightarrow \mathbb{R}^n, \quad (6)$$

where $f_{x_i} := \partial f / \partial x_i$, for each $i = 1, \dots, n$.

- Hessian of f (twice differentiable in Ω):

$$Hf = \nabla^2 f = \left(\frac{\partial^2}{\partial x_j \partial x_i} f \right)_{ij} = \begin{bmatrix} f_{x_1 x_1} & \cdots & f_{x_1 x_n} \\ \vdots & \ddots & \vdots \\ f_{x_n x_1} & \cdots & f_{x_n x_n} \end{bmatrix} : \mathbb{R}^n \rightarrow \mathbb{R}^{n \times n}, \quad (7)$$

where $f_{x_i x_j} := \partial^2 f / (\partial x_j \partial x_i)$, for each $i, j = 1, \dots, n$.

Attention: if $f \in C^2(\Omega)$, $Hf(\mathbf{x})$ is symmetric for each $\mathbf{x} \in \Omega$. Otherwise, symmetry is not guaranteed!

- Jacobian of $\mathbf{F}$ ($f_1, \dots, f_m$ differentiable in Ω):

$$J\mathbf{F} = \left(\frac{\partial}{\partial x_j} f_i \right)_{ij} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1 & \cdots & \frac{\partial}{\partial x_n} f_1 \\ \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m & \cdots & \frac{\partial}{\partial x_n} f_m \end{bmatrix} = \begin{bmatrix} \nabla f_1^T \\ \vdots \\ \nabla f_m^T \end{bmatrix} : \mathbb{R}^n \rightarrow \mathbb{R}^{m \times n}. \quad (8)$$

2.1 Finite Differences (Gradient)

If $f : \Omega \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$ is $C^1(\Omega)$, we can approximate the gradient of f in $\hat{\mathbf{x}} \in \Omega$ with the following expressions.

- Forward Finite Differences (FW-FD):

$$f_{x_i}(\hat{\mathbf{x}}) = \frac{\partial}{\partial x_i} f(\hat{\mathbf{x}}) \approx \frac{f(\hat{\mathbf{x}} + h\mathbf{e}_i) - f(\hat{\mathbf{x}})}{h}, \quad \forall i = 1, \dots, n; \quad (9)$$

- Centered Finite Differences (C-FD):

$$f_{x_i}(\hat{\mathbf{x}}) = \frac{\partial}{\partial x_i} f(\hat{\mathbf{x}}) \approx \frac{f(\hat{\mathbf{x}} + h\mathbf{e}_i) - f(\hat{\mathbf{x}} - h\mathbf{e}_i)}{2h}, \quad \forall i = 1, \dots, n. \quad (10)$$

2.1.1 Choice of h

Large values for h return obviously poor approximations, but too small values are characterized by numerical cancellation problems.

In FW-FD, a suggested trade-off is to select $h = \sqrt{\varepsilon_m}$ (see [2] for motivations), or $h_i = \sqrt{\varepsilon_m} |\hat{x}_i|$ for approximating the partial derivative f_{x_i} . For simplicity, this value can be adopted also for C-FD, even if better values of h exist.

2.2 Finite Differences (Hessian)

If the function $f : \Omega \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$ is $C^2(\Omega)$ (i.e., symmetric Hessian for each $\mathbf{x} \in \Omega$), we can approximate the second order derivatives of f in $\hat{\mathbf{x}} \in \Omega$ with the following expressions.

- *Hessian's diagonal elements:*

$$(Hf)_{ii}(\hat{\mathbf{x}}) = f_{x_i x_i}(\hat{\mathbf{x}}) \approx \frac{f(\hat{\mathbf{x}} + h\mathbf{e}_i) - 2f(\hat{\mathbf{x}}) + f(\hat{\mathbf{x}} - h\mathbf{e}_i)}{h^2}, \quad (11)$$

for each $i = 1 \dots, n$;

- *Hessian's non-diagonal elements:*

$$(Hf)_{ij}(\hat{\mathbf{x}}) = f_{x_i x_j}(\hat{\mathbf{x}}) \approx \frac{f(\hat{\mathbf{x}} + h\mathbf{e}_i + h\mathbf{e}_j) - f(\hat{\mathbf{x}} + h\mathbf{e}_i) - f(\hat{\mathbf{x}} + h\mathbf{e}_j) + f(\hat{\mathbf{x}})}{h^2}, \quad (12)$$

for each $i, j = 1, \dots, n, i \neq j$.

2.2.1 Choice of h

The choice of h for the Hessian, in order to avoid numerical cancellation problems, is suggested to be greater than the h used for the gradient, due to the h^2 in the denominator. Then, for example, we can use an h that is the square root of the value that we would use for the gradient: $h = \sqrt{h_{grad}}$.

2.3 Finite Differences (Jacobian)

If $\mathbf{F} = (f_1, \dots, f_m) : \Omega \subseteq \mathbb{R}^n \rightarrow \mathbb{R}^m$ is such that $f_1, \dots, f_m \in C^1(\Omega)$, we can approximate the Jacobian of $\mathbf{F}$ in $\hat{\mathbf{x}} \in \Omega$ with the following expressions.

- *Forward Finite Differences (j -th Jacobian's column):*

$$(J\mathbf{F})_{\cdot j}(\hat{\mathbf{x}}) \approx \frac{\mathbf{F}(\hat{\mathbf{x}} + h\mathbf{e}_j) - \mathbf{F}(\hat{\mathbf{x}})}{h}, \quad \forall j = 1, \dots, n; \quad (13)$$

- *Centered Finite Differences (j -th Jacobian's column):*

$$(J\mathbf{F})_{\cdot j}(\hat{\mathbf{x}}) \approx \frac{\mathbf{F}(\hat{\mathbf{x}} + h\mathbf{e}_j) - \mathbf{F}(\hat{\mathbf{x}} - h\mathbf{e}_j)}{2h}, \quad \forall j = 1, \dots, n; \quad (14)$$

Attention: remember that is possible to approximate more columns at the same times if $J\mathbf{F}$ is sparse (see [2]).

2.4 Exercises

The text of the exercises is for Matlab users. Nonetheless, you can use Python instead.

Exercise 2.1 (Finite Differences - Gradient). *Write a Matlab function finddiff_grad.m that implements both the forward and centered finite difference methods for the gradient computation, given:*

- f : a function handle variable that, for each column vector $\mathbf{x} \in \mathbb{R}^n$, returns $f(\mathbf{x})$, where $f : \Omega \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$ is the function whose gradient we want to calculate;
- $\mathbf{x}$: a column vector of n elements representing the point where we want to compute the gradient of f ;
- h : the finite difference “step”;

- **type**: a string of characters such that if it is 'fw', the function uses the forward method (default), and if it is 'c', the function uses the centered method.

The output (i.e., the gradient approximation) must be stored as a column vector.

Once you have written the function, test it using the functions in Section B.1 and compare the results with the exact gradients.

Exercise 2.2 (Finite Differences - Jacobian). Write a Matlab function `findiff_J.m` that implements both the forward and centered finite difference methods for the Jacobian computation, given:

- **F**: a function handle variable that, for each column vector $\mathbf{x} \in \mathbb{R}^n$, returns $\mathbf{F}(\mathbf{x})$, where $\mathbf{F} : \Omega \subseteq \mathbb{R}^n \rightarrow \mathbb{R}^m$ is the function whose Jacobian we want to calculate;
- **x**: a column vector of n elements representing the point where we want to compute the Jacobian of $\mathbf{F}$;
- **h**: the finite difference “step”;
- **type**: a string of characters such that if it is 'fw', the function uses the forward method (default), and if it is 'c', the function uses the centered method.

Once you have written the function, test it using the functions in Section B.1 for building a vectorial test function $\mathbf{F}$ and compare the results with the exact gradients.

Exercise 2.3 (Finite Differences - Hessian). Write a Matlab function `findiff_Hess.m` that implements the finite difference method for the Hessian computation (symmetric case only), given:

- **f**: a function handle variable that, for each column vector $\mathbf{x} \in \mathbb{R}^n$, returns $f(\mathbf{x})$, where $f : \Omega \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$ is the C^2 function whose Hessian we want to calculate;
- **x**: a column vector of n elements representing the point where we want to compute the gradient of f ;
- **h**: the finite difference “step”.

Once you have written the function, test it using the functions in Section B.1 and compare the results with the exact Hessians.

Exercise 2.4 (Finite Differences - Hessian as Jacobian of the Gradient). Modify the Matlab function `findiff_J.m` such that it can force the symmetry in the output matrix.

Once you have written the function, test it using the functions in Section B.1 for computing the Hessians, given the exact gradients; then, compare the results with the exact Hessians.

Exercise 2.5 (Finite Differences - Approximation Quality and h). Write a Matlab script `lab-02_FD_tests.m` where you compute the gradient, Jacobian, and Hessian of some test functions, varying the value of h . Analyze and comment on the results.

Exercise 2.6 (Finite Differences - Smart Implementations). Look at the test function illustrated in B.2. Then, write a Matlab custom functions for approximating efficiently with Finite Differences the gradient and the Hessian (both directly or as Jacobian of the exact gradient).

Suggestion: exploit the summation structure for the gradient and the sparsity for the Hessian.

3 Iterative Methods for Linear Systems

As we stated at the end of the Section 1, direct methods are not particularly suitable for linear systems with very large sparse matrix, due to the *fill-in* phenomenon (see Section 1.7). On the contrary, *iterative methods* can preserve the advantages given by a sparse matrix.

The main idea behind an iterative method is the following procedure:

- Choose a *starting guess* $\mathbf{x}^{(0)}$ for the solution of the problem;
- Build a sequence $\{\mathbf{x}^{(k)}\}_{k \in \mathbb{N}}$ such that:
 - $\mathbf{x}^{(k+1)}$ is computed using information of step k (or even previous steps);
 - $\mathbf{x}^{(k)} \xrightarrow{k \rightarrow \infty} \mathbf{x}^*$, where $\mathbf{x}^*$ is an exact solution of the problem.

Since the sequence of vectors $\{\mathbf{x}^{(k)}\}_{k \in \mathbb{N}}$ is infinite, the introduction of stopping criteria is necessary for returning an approximated solution $\hat{\mathbf{x}} \approx \mathbf{x}^*$. Concerning the methods for linear systems there can be:

- **Relative Stopping Criteria:**

- *Relative Increment*: $\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\| / \|\mathbf{x}^{(k+1)}\| < \tau$;
- *Relative Residual*: $\|\mathbf{r}^{(k+1)}\| / \|\mathbf{b}\| < \tau$, where $\mathbf{r}^{(k)} := \mathbf{b} - \mathbf{A}\mathbf{x}^{(k)}$ is the *residual* for the “guess” $\mathbf{x}^{(k)}$, $k \in \mathbb{N}$;
- ...

- **Absolute Stopping Criteria:**

- *Absolute Increment*: $\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\| < \tau$;
- *Absolute Residual*: $\|\mathbf{r}^{(k+1)}\| < \tau$;
- ...

Tolerance and Stopping Criteria: the value τ used above for describing the stopping criteria is known as *tolerance* and it is a positive real value ($\tau > 0$), usually very small (i.e., $\tau \approx 0$). Indeed, the smaller the relative/absolute quantities at step $k + 1$, the more we can *assume* $\mathbf{x}^{(k+1)} \approx \mathbf{x}^*$.

In the end of this introduction, concerning the iterative methods for linear systems we can observe that:

- There is not a best stopping criterion. The best choice depends on the problem characteristics;
- The smaller tolerance τ , the larger the number of iterations required. On the other hand, the smaller the tolerance, the higher the accuracy in approximating $\mathbf{x}^*$;
- Ill-conditioning has effects also on iterative methods. Indeed, at step k , the inequality (5) can be translated into:

$$\frac{\|\mathbf{x}^{(k)} - \mathbf{x}^*\|}{\|\mathbf{x}^{(k)}\|} \leq k(A) \frac{\|\mathbf{r}^{(k)}\|}{\|\mathbf{b}\|}. \quad (15)$$

Then, even if we use a relative residual stopping criteria with τ very small ($\|\mathbf{r}^{(k)}\| / \|\mathbf{b}\|$), we only know that $\|\mathbf{x}^{(k)} - \mathbf{x}^*\| / \|\mathbf{x}^{(k)}\| \leq k(A)\tau$.

3.1 The Gradient Method

Let us consider a square linear system like (1), where A is *symmetric* and positive definite (see Section 1.1); necessarily, A is non-singular. Let $J : \mathbb{R}^n \rightarrow \mathbb{R}$ be a quadratic function such that

$$J(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T A \mathbf{x} - \mathbf{x}^T \mathbf{b}. \quad (16)$$

Therefore, J is a (quadratic) strictly convex function defined on $\mathbb{R}^n$ and it has a unique (global) minimum in $\mathbf{x}^*$. Now we recall the following theorems.

Theorem 3.1 (First Order Necessary Conditions [1]). *Let $\mathbf{x}^* \in \mathbb{R}^n$ and let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be such that it is continuously differentiable in an open neighborhood of $\mathbf{x}^*$. If $\mathbf{x}^*$ is a local minimizer of f , then $\nabla f(\mathbf{x}^*) = \mathbf{0}$.*

Theorem 3.2 (Second Order Necessary Conditions [1]). *Let $\mathbf{x}^* \in \mathbb{R}^n$ and let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be such that its Hessian $Hf : \mathbb{R}^n \rightarrow \mathbb{R}^{n \times n}$ exists in $\mathbf{x}^*$ and is continuous in an open neighborhood of $\mathbf{x}^*$. If $\mathbf{x}^*$ is a local minimizer of f , then $\nabla f(\mathbf{x}^*) = \mathbf{0}$ and $Hf(\mathbf{x}^*)$ is positive semi-definite.*

Theorem 3.3 (Second Order Sufficient Conditions [1]). *Let $\mathbf{x}^* \in \mathbb{R}^n$ and let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be such that its Hessian Hf exists in $\mathbf{x}^*$ and is continuous in an open neighborhood of $\mathbf{x}^*$. If $\nabla f(\mathbf{x}^*) = \mathbf{0}$ and $Hf(\mathbf{x}^*)$ is positive definite, then $\mathbf{x}^*$ is a strict local minimizer of f .*

Now, we observe that $\nabla J(\mathbf{x}) = A\mathbf{x} - \mathbf{b}$ and $H_J(\mathbf{x}) = A$, for each $\mathbf{x} \in \mathbb{R}^n$. Therefore, we have that the global minimizer $\mathbf{x}^*$ of J is also the solution of the linear system $A\mathbf{x} = \mathbf{b}$, indeed:

- $\mathbf{x}^*$ minimizer of $J \Rightarrow \nabla J(\mathbf{x}^*) = A\mathbf{x}^* - \mathbf{b} = \mathbf{0}$ (Theorem 3.1);
- $\mathbf{x}^*$ solution of $A\mathbf{x} = \mathbf{b}$ and A symmetric positive definite $\Rightarrow \mathbf{x}^*$ (strict) minimizer of J (Theorem 3.3);
- A is non-singular $\Rightarrow A\mathbf{x} = \mathbf{b}$ has a unique solution;
- J is strictly convex $\Rightarrow$ has a unique minimizer.

Main Idea: for finding the solution of the linear system, we find instead the minimizer of J with an iterative method, instead of using a direct method on the linear system. For doing so, we recall the definition of descent direction (see Definition 3.1 below).

Definition 3.1 (Descent Direction). *A vector $\mathbf{p} \in \mathbb{R}^n$ is a descent direction for a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ at $\mathbf{x} \in \mathbb{R}^n$ if exists $\varepsilon > 0$ such that*

$$f(\mathbf{x} + \alpha \mathbf{p}) < f(\mathbf{x}), \quad (17)$$

for each $0 < \alpha \leq \varepsilon$. Moreover, let f be continuously differentiable at $\mathbf{x}$; then, $\mathbf{p}$ is a descent direction for f at $\mathbf{x}$ if and only if

$$\nabla f(\mathbf{x})^T \mathbf{p} < 0. \quad (18)$$

See Figure 3 for an example in $\mathbb{R}^2$.

Remark 3.1 (Steepest Descent/Ascent Direction). Given Definition 3.1, we observe that $\nabla f(\mathbf{x})$ and $-\nabla f(\mathbf{x})$ are the directions of steepest ascent and descent, respectively, for f in $\mathbf{x}$.

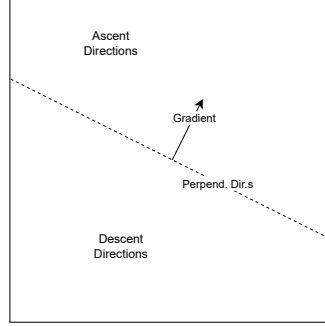


Figure 3: Example of regions of ascent, perpendicular, and descent directions in $\mathbb{R}^2$, with respect to a given gradient vector.

3.1.1 Sequence of the Gradient Method

Gathering all the knowledge described above, the iterative *gradient method* used for solving the linear system (i.e., minimizing J) consists in building the following sequence:

$$\begin{cases} \mathbf{x}^{(0)} \in \mathbb{R}^n, & \text{given} \\ \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha^{(k)} \nabla J(\mathbf{x}^{(k)}), & \forall k \geq 0 \end{cases}, \quad (19)$$

where $\alpha^{(k)} \in \mathbb{R}_+$ is the best step length for reaching the minimum available value for $J(\mathbf{x}^{(k+1)})$.

In particular, denoting by $\mathbf{r}^{(k)}$ the k -th residual, i.e.

$$\mathbf{r}^{(k)} := \mathbf{b} - A\mathbf{x}^{(k)}, \forall k \geq 0, \quad (20)$$

we have that $\mathbf{r}^{(k)} \equiv -\nabla J(\mathbf{x}^{(k)})$ and, therefore, the sequence can be rewritten as

$$\begin{cases} \mathbf{x}^{(0)} \in \mathbb{R}^n, & \text{given} \\ \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{r}^{(k)}, & \forall k \geq 0 \end{cases}, \quad (21)$$

where $\alpha^{(k)} = (\mathbf{r}^{(k)T} \mathbf{r}^{(k)}) / (\mathbf{r}^{(k)T} A \mathbf{r}^{(k)})$, because $\frac{d}{d\alpha} J(\mathbf{x} + \alpha \mathbf{r}) = \dots = -\mathbf{r}^T \mathbf{r} + \alpha \mathbf{r}^T A \mathbf{r}$.

In the following, we report two pseudo-codes for the implementation of the gradient method.

Algorithm 3.1 (Gradient Method (naive)).

```

1:  $\mathbf{x}^{(0)}$  given
2:  $k \leftarrow 0$ 
3: while stopping criteria are not satisfied do
4:    $\mathbf{r}^{(k)} \leftarrow \mathbf{b} - A\mathbf{x}^{(k)}$ 
5:    $\alpha^{(k)} \leftarrow (\mathbf{r}^{(k)T} \mathbf{r}^{(k)}) / (\mathbf{r}^{(k)T} A \mathbf{r}^{(k)})$ 
6:    $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{r}^{(k)}$ 
7:    $k \leftarrow k + 1$ 
8: end while
9:  $\hat{\mathbf{x}} \leftarrow \mathbf{x}^{(k)}$ 
10: return :  $\hat{\mathbf{x}}$ 
```

Algorithm 3.2 (Gradient Method).

```

1:  $\mathbf{x}^{(0)}$  given
2:  $\mathbf{r}^{(0)} \leftarrow \mathbf{b} - A\mathbf{x}^{(0)}$ 
3:  $k \leftarrow 0$ 
4: while stopping criteria are not satisfied do
```



```

5:    $\mathbf{z}^{(k)} \leftarrow A\mathbf{r}^{(k)}$ 
6:    $\alpha^{(k)} \leftarrow (\mathbf{r}^{(k)T} \mathbf{r}^{(k)}) / (\mathbf{r}^{(k)T} \mathbf{z}^{(k)})$ 
7:    $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{r}^{(k)}$ 
8:    $\mathbf{r}^{(k+1)} \leftarrow \mathbf{r}^{(k)} - \alpha^{(k)} \mathbf{z}^{(k)}$   $\triangleright$  because  $\mathbf{r}^{(k+1)} = \mathbf{b} - A(\mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{r}^{(k)})$ 
9:    $k \leftarrow k + 1$ 
10: end while
11:  $\hat{\mathbf{x}} \leftarrow \mathbf{x}^{(k)}$ 
12: return :  $\hat{\mathbf{x}}$ 

```

3.1.2 Convergence Properties

The convergence of the gradient method is characterized by the following property:

$$\|\mathbf{x}^{(k)} - \mathbf{x}^*\|_A \leq 2 \left(\frac{k_2(A) - 1}{k_2(A) + 1} \right)^k \|\mathbf{x}^{(0)} - \mathbf{x}^*\|_A, \quad \forall k \geq 0, \quad (22)$$

where $\|\cdot\|_A$ is the *energy norm* such that $\|\mathbf{x}\|_A := \sqrt{\mathbf{x}^T A \mathbf{x}}$, and $k_2(A) := \|A\|_2 \|A^{-1}\|_2 = \lambda_{\max}/\lambda_{\min}$ (λ denotes eigenvalues of A).

Observation: the more ill-conditioned A , the larger is the upper bound of $\|\mathbf{x}^{(k)} - \mathbf{x}^*\|_A$ and, therefore, the slower can be the convergence.

3.2 Conjugate Gradient Method

The idea behind the *conjugate gradient* (CG) method is to modify the gradient method using a different descent direction, other than $\mathbf{r} = -\nabla J(\mathbf{x})$, for improving the convergence.

Without going into details, the CG method uses as descent directions the sequence

$$\begin{cases} \mathbf{p}^{(0)} = \mathbf{r}^{(0)} \\ \mathbf{p}^{(k+1)} = \mathbf{r}^{(k+1)} + \beta^{(k+1)} \mathbf{p}^{(k)}, \quad \forall k \geq 0 \end{cases}, \quad (23)$$

where $\beta^{(k+1)} = -(\mathbf{p}^{(k)T} A \mathbf{r}^{(k+1)}) / (\mathbf{p}^{(k)T} A \mathbf{p}^{(k)})$. Then, the sequence build by the CG for finding $\mathbf{x}^*$ is

$$\begin{cases} \mathbf{x}^{(0)} \in \mathbb{R}^n, & \text{given} \\ \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{p}^{(k)}, & \forall k \geq 0 \end{cases}, \quad (24)$$

where $\alpha^{(k)} = (\mathbf{r}^{(k)T} \mathbf{p}^{(k)}) / (\mathbf{p}^{(k)T} A \mathbf{p}^{(k)})$.

The advantages and disadvantages of CG method are in the following list:

- + *Finite termination:* the CG method reaches the solution $\mathbf{x}^* \in \mathbb{R}^n$ in at most n steps;
- if $n \gg 1$, the advantages of the finite termination can be lost;
- Due to arithmetic errors, exact conjugacy can be lost;
- ± Even losing the exact conjugacy, we can implement CG as an iterative method with stopping criteria.

In the following, we report two pseudo-codes for the implementation of the CG method.

Algorithm 3.3 (Conjugate Gradient Method (naive)).

```

1:  $\mathbf{x}^{(0)}$  given
2:  $\mathbf{r}^{(0)} \leftarrow \mathbf{b} - A\mathbf{x}^{(0)}$ 
3:  $\mathbf{p}^{(0)} \leftarrow \mathbf{r}^{(0)}$ 
4:  $k \leftarrow 0$ 

```

```

5: while stopping criteria are not satisfied do
6:    $\alpha^{(k)} \leftarrow (\mathbf{r}^{(k)T} \mathbf{p}^{(k)}) / (\mathbf{p}^{(k)T} A \mathbf{p}^{(k)})$ 
7:    $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{p}^{(k)}$ 
8:    $\mathbf{r}^{(k+1)} \leftarrow \mathbf{b} - A \mathbf{x}^{(k+1)}$ 
9:    $\beta^{(k+1)} \leftarrow -(\mathbf{p}^{(k)T} A \mathbf{r}^{(k+1)}) / (\mathbf{p}^{(k)T} A \mathbf{p}^{(k)})$ 
10:   $\mathbf{p}^{(k+1)} \leftarrow \mathbf{r}^{(k+1)} + \beta^{(k+1)} \mathbf{p}^{(k)}$ 
11:   $k \leftarrow k + 1$ 
12: end while
13:  $\hat{\mathbf{x}} \leftarrow \mathbf{x}^{(k)}$ 
14: return :  $\hat{\mathbf{x}}$ 

```

Algorithm 3.4 (Conjugate Gradient Method).

```

1:  $\mathbf{x}^{(0)}$  given
2:  $\mathbf{r}^{(0)} \leftarrow \mathbf{b} - A \mathbf{x}^{(0)}$ 
3:  $\mathbf{p}^{(0)} \leftarrow \mathbf{r}^{(0)}$ 
4:  $k \leftarrow 0$ 
5: while stopping criteria are not satisfied do
6:    $\mathbf{z}^{(k)} \leftarrow A \mathbf{p}^{(k)}$ 
7:    $\alpha^{(k)} \leftarrow (\mathbf{r}^{(k)T} \mathbf{p}^{(k)}) / (\mathbf{p}^{(k)T} \mathbf{z}^{(k)})$ 
8:    $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{p}^{(k)}$ 
9:    $\mathbf{r}^{(k+1)} \leftarrow \mathbf{r}^{(k)} - \alpha^{(k)} \mathbf{z}^{(k)}$ 
10:   $\beta^{(k+1)} \leftarrow -(\mathbf{r}^{(k+1)T} \mathbf{z}^{(k)}) / (\mathbf{p}^{(k)T} \mathbf{z}^{(k)})$ 
11:   $\mathbf{p}^{(k+1)} \leftarrow \mathbf{r}^{(k+1)} + \beta^{(k+1)} \mathbf{p}^{(k)}$ 
12:   $k \leftarrow k + 1$ 
13: end while
14:  $\hat{\mathbf{x}} \leftarrow \mathbf{x}^{(k)}$ 
15: return :  $\hat{\mathbf{x}}$ 

```

3.2.1 Preconditioning

The convergence of the CG is characterized by the following property:

$$\|\mathbf{x}^{(k)} - \mathbf{x}^*\|_A \leq 2 \left(\frac{\sqrt{\kappa_2(A)} - 1}{\sqrt{\kappa_2(A)} + 1} \right)^k \|\mathbf{x}^{(0)} - \mathbf{x}^*\|_A, \quad \forall k \geq 0, \quad (25)$$

where $\|\cdot\|_A$ is the *energy norm* such that $\|\mathbf{x}\|_A := \sqrt{\mathbf{x}^T A \mathbf{x}}$, and $\kappa_2(A) := \|A\|_2 \|A^{-1}\|_2 = \lambda_{\max}/\lambda_{\min}$ (λ denotes eigenvalues of A).

Observation: the more ill-conditioned A , the larger is the upper bound of $\|\mathbf{x}^{(k)} - \mathbf{x}^*\|_A$ and, therefore, the slower can be the convergence. Nonetheless, the upper bound in (25) is lower than the one in (22).

Given the above observation, we observe that a method to reduce the effects of an ill-conditioned matrix exists. The main idea is to introduce a variable substitution

$$\mathbf{x} = C^{-1} \mathbf{y}. \quad (26)$$

Using (26), we have that (16) changes into $J(\mathbf{y}) = \frac{1}{2} \mathbf{y}^T C^{-T} A C^{-1} \mathbf{y} - \mathbf{y}^T C^{-T} \mathbf{b}$, and we look for the solution of the linear system $C^{-T} A C^{-1} \mathbf{y} = C^{-T} \mathbf{b}$. Therefore, with a proper choice of C we can reduce the condition number of the new matrix and speed up the convergence of the CG method.

Typically, the “best” *preconditioner* (i.e., the matrix C) does not exist. In general, it depends on the nature of the matrix A . Nonetheless, a canonical choice can be the matrix $\tilde{L}^T$ returned by the *incomplete Cholesky* method.

The incomplete Cholesky method performs an incomplete Cholesky factorization such that it returns a *sparse* lower triangular matrix $\tilde{L}$ such that $\tilde{L}^T \approx R$, where R is the upper triangular

matrix of the Cholesky factorization (see Section 1.1). The choice of $C = \tilde{L}^T$ is motivated by the fact that the more $\tilde{L}^T \approx R$, the more $C^{-T}AC^{-1} \approx \mathbb{I}_n$ and, therefore, the problem is more stable, because:

$$\mathbb{I}_n R\mathbf{x} \approx C^{-T}AC^{-1}\mathbf{y} = C^{-T}\mathbf{b} \approx R^{-T}\mathbf{b} \quad (27)$$

Of course, the more $\tilde{L}^T \approx R$, the more the density of $\tilde{L}$, losing the advantages of sparsity.

Such kind of preconditioning is called *two-sided* or *split* preconditioning (there are also *left/right* preconditioning methods in the literature).

3.3 Other Iterative Methods

We point the attention of the reader on the characteristics of the matrix A we used in this section. We have always assumed A *symmetric* and *positive definite* (at most, we can relax the hypotheses to *positive semi-definiteness*). Then, if A does not satisfy these properties, the gradient and CG methods cannot be applied.

Other iterative methods, can be for example the *generalized minimal residual method* (GMRES), where the preconditioning can be performed exploiting an incomplete LU factorization.

3.4 Exercises

From now on, we will write the text of the exercises for Matlab users. Nonetheless, you can use Python instead.

Exercise 3.1 (Gradient Method). Write a function called `gradient_linsys` that implement in Matlab the Algorithm 3.2 using a relative residual as stopping criterium.

Test your function on the linear system defined by the variables inside the file `lab03_sparse_linsys.mat`.

Exercise 3.2 (Conjugate Gradient Method). Write a function called `cg_linsys` that implement in Matlab the Algorithm 3.4 using a relative residual as stopping criterium.

Test your function on the linear system defined by the variables inside the file `lab03_sparse_linsys.mat`.

Exercise 3.3. Write a script where:

1. Generate two sparse tridiagonal matrices (see `spdiags` function) $A_1, A_2 \in \mathbb{R}^{n \times n}$ (say $n = 1000$) such that

$$A_i = \begin{bmatrix} \alpha_i & -1 & & & \\ -1 & \ddots & \ddots & & \\ & \ddots & \ddots & \ddots & \\ & & \ddots & \ddots & -1 \\ & & & -1 & \alpha_i \end{bmatrix}, \quad \alpha_1 = 4, \alpha_2 = 2.$$

2. Compute and visualize $k_2(A_1), k_2(A_2)$;
3. For each $\tau = 10^{-4}, \dots, 10^{-10}$, solve the linear systems $A_i \mathbf{x} = \mathbf{b}_i$, with exact solution $\mathbf{x}_i^* = (1, \dots, 1) \in \mathbb{R}^n$, $i = 1, 2$ (i.e., build $\mathbf{b}_i$ as in Exercise 1.1, item 3).

In particular, solve the linear systems with `gradient_linsys` of Exercise 3.1, with `cg_linsys` of Exercise 3.2, and with the `pcg` matlab function, both with and without preconditioning (see `ichol` function).

4. *Plot the number of iterations used by the methods for solving the linear systems, varying the tolerance values.*

4 Steepest Descent

Let the function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be given. The Steepest Descent (SD) method is an iterative optimization method that can be interpreted as the nonlinear generalization of the Gradient method; SD starts from a given vector $\mathbf{x}^{(0)} \in \mathbb{R}^n$, and computes a sequence of vectors $\{\mathbf{x}^{(k)}\}_{k \in \mathbb{N}}$ characterized by

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{p}^{(k)}, \quad \forall k \geq 0, \quad (28)$$

where the descent direction $\mathbf{p}^{(k)}$ is the steepest one, i.e. $\mathbf{p}^{(k)} = -\nabla f(\mathbf{x}^{(k)})$, and the step length factor $\alpha \in \mathbb{R}^+$.

Attention: Contrary to the Gradient method, we have not a formula for computing the optimal step length $\alpha^{(k)}$ (because f is a generic differentiable function). The choice of $\alpha^{(k)}$ is problem-dependent and different strategies exist to select the optimal one.

N.B.: In this laboratory, we will use a fixed value $\alpha = \alpha^{(k)}$ for each step.

4.1 Exercises

The text of the exercises is for Matlab users. Nonetheless, you can use Python instead.

Exercise 4.1 (Steepest Descent). *Write a Matlab function `steepest_descent.m` that implements the steepest descent optimization method, given:*

- **`x0`**: a column vector of n elements representing the starting point for the optimization method;
- **`f`**: a function handle variable that, for each column vector $\mathbf{x} \in \mathbb{R}^n$, returns the value $f(\mathbf{x})$, where $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is the loss function that have to be minimized;
- **`gradf`**: a function handle variable that, for each column vector $\mathbf{x} \in \mathbb{R}^n$, returns the value $\nabla f(\mathbf{x})$ as a column vector, where $\nabla f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is the gradient of f ;
- **`alpha`**: a real scalar value characterizing the step length of the optimization method (fixed, for simplicity);
- **`kmax`**: an integer scalar value characterizing the maximum number of iterations of the method;
- **`tolgrad`**: a real scalar value characterizing the tolerance with respect to the norm of the gradient in order to stop the method.

The outputs of the function must be:

- **`xk`**: the last vector $\mathbf{x}^{(k)} \in \mathbb{R}^n$ computed by the optimization method before it stops;
- **`fk`**: the value $f(\mathbf{x}^{(k)})$;
- **`gradfk_norm`**: the euclidean norm of $\nabla f(\mathbf{x}^{(k)})$;
- **`k`**: index value of the last step executed by the optimization method before stopping;
- **`xseq`**: a matrix/vector in $\mathbb{R}^{n \times k}$ such that each column j is the j -th vector $\mathbf{x}^{(j)} \in \mathbb{R}^n$ generated by the iterations of the method.

Once you have written the function, test it using the data `x0`, `f`, `gradf`, `alpha`, `kmax`, `tolgrad`, inside the file `test_functions2.mat`

Exercise 4.2 (Steepest Descent Path Visualization). *Given the function of the previous exercise, write a Matlab script where you test this function using the 2-dimensional functions inside the file `test_functions2.mat` (see Section B.1). In particular, for each 2-dimensional function in the `.mat` file:*

- *run the SD procedure by varying $\alpha = 1e-1, 1e-2, 1e-3, 1e-4$;*
- *repeat the previous item, but using the Finite Differences (of your choice) to compute the gradient of the functions, varying h among the values $h = 1e-8, 1e-6, 1e-4$;*
- *For each procedure generated with the previous two steps, collect:*
 - *the number of steps for reaching the convergence (or a stopping criterion);*
 - *the norm of the gradient at the last step;*
 - *the coordinates of the minimizer $\hat{\mathbf{x}}$ (just because we are in 2D) and the value $f(\hat{\mathbf{x}})$;*
- *plot a top view of the loss f using the Matlab function `contour`, together with the sequence $\mathbf{xseq}$ in $\mathbb{R}^2$;*
- *the surface of the loss f using the Matlab function `surf`, together with the function values of the sequence $\mathbf{xseq}$ in $\mathbb{R}^3$ (i.e., coordinates plus function values). See the `meshgrid` function for preparing the plot domain. You can use the function `f_meshgrid` for evaluating the function on the output matrices of `meshgrid`. For any doubts, see <https://www.mathworks.com/help/matlab/ref/surf.html>.*

Exercise 4.3 (n -Dimensional Steepest Descent). *Repeat the previous exercise (except for the plots) with respect to the n -dimensional function in `test_funcs_ndim.mat`. Run the test for different values of n , e.g.: $n = 10, 100, 1000$.*

5 Steepest Descent with Backtracking

Let the function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be given. In the previous laboratory we have seen that the steepest descent method is an iterative optimization method that, starting from a given vector $\mathbf{x}^{(0)} \in \mathbb{R}^n$, computes a sequence of vectors $\{\mathbf{x}^{(k)}\}_{k \in \mathbb{N}}$ characterized by

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha \mathbf{p}^{(k)}, \quad \forall k \geq 0, \quad (29)$$

where the descent direction $\mathbf{p}^{(k)}$ is the steepest one, i.e. $\mathbf{p}^{(k)} = -\nabla f(\mathbf{x}^{(k)})$, and the step length factor $\alpha \in \mathbb{R}^+$.

5.1 Backtracking

Let the function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be given. The backtracking strategy for an iterative optimization method consists of looking for a value $\alpha^{(k)}$ satisfying the Armijo condition at each step k of the method, i.e.

$$f(\underbrace{\mathbf{x}^{(k+1)}}_{\mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{p}^{(k)}}) \leq f(\mathbf{x}^{(k)}) + c_1 \alpha^{(k)} \nabla f(\mathbf{x}^{(k)})^T \mathbf{p}^{(k)}, \quad (30)$$

where $c_1 \in (0, 1)$ (typically, the standard choice is $c_1 = 10^{-4}$).

We recall that the Armijo condition suggests that a “good” $\alpha^{(k)}$ is such that you have a sufficient decrease in f and, moreover, the function value at the new point $f(\mathbf{x}^{(k+1)}) = f(\mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{p}^{(k)})$ is under the “reduced tangent hyperplane” of f at $\mathbf{x}^{(k)}$.

To better explain the Armijo condition, we look at the function $\phi(\alpha) := f(\mathbf{x}^{(k)} + \alpha \mathbf{p}^{(k)})$, such that $\phi'(\alpha) = \nabla f(\mathbf{x}^{(k)} + \alpha \mathbf{p}^{(k)})^T \mathbf{p}^{(k)}$ and $\phi'(0) = \nabla f(\mathbf{x}^{(k)})^T \mathbf{p}^{(k)}$ (see Figure 4).

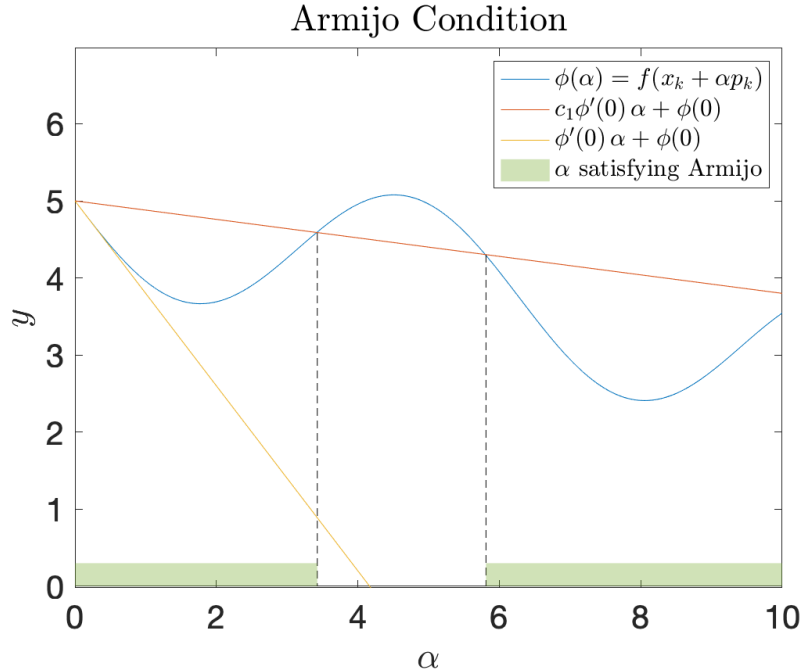


Figure 4: Example of the Armijo condition.

The backtracking strategy is an *iterative process* that looks for this value $\alpha^{(k)}$. Given a factor $\rho \in (0, 1)$ and a starting value $\alpha_0^{(k)}$ for $\alpha^{(k)}$, we decrease iteratively $\alpha_0^{(k)}$, multiplying it by ρ , until the Armijo condition is satisfied. Then $\alpha^{(k)} = \rho^{t_k} \alpha_0^{(k)}$, for a $t_k \in \mathbb{N}$, if it satisfies Armijo but $\rho^{t_k-1} \alpha_0^{(k)}$ does not.

Remark 5.1 (Few things to keep in mind).

1. The Armijo condition is always satisfied for extremely small values of α . Then, it is not enough to ensure that the algorithm makes reasonable progress; indeed, if α is too small, unacceptably short steps are taken (see item 3 below).
2. For simplicity, we consider ρ as a fixed parameter, but it can be chosen using already available information, changing with the iterations;
3. Other conditions could be imposed to guarantee that not too-short steps are taken (e.g., Wolfe conditions¹), but they are not practical to be implemented. Practical implementations, instead of imposing a second condition, frequently use the backtracking strategy; for example, a proper choice of $\alpha_0^{(k)}$, ρ , and the maximum number T of backtracking steps can guarantee that $\rho^T \alpha_0^{(k)} \geq \epsilon$, where ϵ is the minimum step-length acceptable.
4. The choice of $\alpha_0^{(k)}$ is problem-dependent and/or method-dependent.

5.2 Exercises

The text of the exercises is for Matlab users. Nonetheless, you can use Python instead.

Exercise 5.1 (Steepest Descent with Backtracking). *Write a Matlab function `steepest_desc_bcktrck.m` that implements the steepest descent optimization method with the backtracking strategy, given:*

- ***x0***: a column vector of n elements representing the starting point for the optimization method;
- ***f***: a function handle variable that, for each column vector $\mathbf{x} \in \mathbb{R}^n$, returns the value $f(\mathbf{x})$, where $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is the loss function that have to be minimized;
- ***gradf***: a function handle variable that, for each column vector $\mathbf{x} \in \mathbb{R}^n$, returns the value $\nabla f(\mathbf{x})$ as a column vector, where $\nabla f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is the gradient of f ;
- ***alpha0***: a real scalar value characterizing the starting value for the backtracking (fixed, for simplicity);
- ***kmax***: an integer scalar value characterizing the maximum number of iterations of the method;
- ***tolgrad***: a real scalar value characterizing the tolerance with respect to the norm of the gradient in order to stop the method.
- ***c1***: the factor c_1 for the Armijo condition that must be a scalar in $(0, 1)$;
- ***rho***: factor less than 1, used to reduce α (fixed, for simplicity);
- ***btmax***: maximum number of steps allowed to update α during the backtracking strategy.

The outputs of the function must be:

- ***xk***: the last vector $\mathbf{x}^{(k)} \in \mathbb{R}^n$ computed by the optimization method before it stops;
- ***fk***: the value $f(\mathbf{x}^{(k)})$;
- ***gradfk_norm***: the euclidean norm of $\nabla f(\mathbf{x}^{(k)})$;

¹i.e., Armijo condition and curvature condition $\nabla f(\mathbf{x}_{k+1})^T \mathbf{p}_k \geq c_2 \nabla f(\mathbf{x}_k)^T \mathbf{p}_k$, $c_2 \in (c_1, 1)$.

- **k**: index value of the last step executed by the optimization method before stopping;
- **xseq**: a matrix/vector in $\mathbb{R}^{n \times k}$ such that each column j is the j -th vector $\mathbf{x}^{(j)} \in \mathbb{R}^n$ generated by the iterations of the method.
- **btseq**: row vector in $\mathbb{R}^k$ such that the j -th element is the number of backtracking iterations done at the j -th step of the steepest descent.

Extra options:

- implement the procedure such that it stops if the backtracking strategy cannot find $\alpha^{(k)}$ satisfying the Armijo condition;
- **flag**: an extra output variable for the function containing a string that is a message. The message explains the result (e.g. "Procedure stopped in k steps, with gradient norm ε " and/or "Procedure stopped because...").

Once you have written the function, test it using the data **x0**, **f**, **gradf**, **alpha0**, **kmax**, **tolgrad**, **c1**, **rho**, **btmax**, inside the file **test_functions2.mat**

Suggestion: copy the code of the function of Exercise 4.1 and modify it adding the backtracking.

Exercise 5.2 (Steepest Descent Path Visualization). Given the function of the previous exercise, write a Matlab script where you test this function using the 2-dimensional functions inside the file **test_functions2.mat** (see Section B.1). In particular, for each 2-dimensional function in the .mat file:

- run the SD + backtracking procedure by varying $\rho = 0.9, 0.75, 0.5, 0.25, 0.1$, with fixed $\alpha_0^{(k)} = 1$ and $c_1 = 1e-4$;
- repeat the previous item, but using the Finite Differences (of your choice) to compute the gradient of the functions, varying h among the values $h = 1e-8, 1e-6, 1e-4$;
- For each procedure generated with the previous two steps, collect:
 - the number of steps for reaching the convergence (or a stopping criterion);
 - the norm of the gradient at the last step;
 - the coordinates of the minimizer $\hat{\mathbf{x}}$ (just because we are in 2D) and the value $f(\hat{\mathbf{x}})$;
- plot a top view of the loss f using the Matlab function **contour**, together with the sequence **xseq** in $\mathbb{R}^2$;
- the surface of the loss f using the Matlab function **surf**, together with the function values of the sequence **xseq** in $\mathbb{R}^3$ (i.e., coordinates plus function values). See the **meshgrid** function for preparing the plot domain. You can use the function **f_meshgrid** for evaluating the function on the output matrices of **meshgrid**. For any doubts, see <https://www.mathworks.com/help/matlab/ref/surf.html>.
- Make a comparison of the results obtained in this exercise with the results obtained in Exercise 4.2.

Exercise 5.3 (n -Dimensional Steepest Descent). Repeat the previous exercise (except for the plots) with respect to the n -dimensional function in **test_funcs_ndim.mat**. Run the test for different values of n , e.g.: $n = 10, 100, 1000$. Make a comparison of the results obtained in this exercise with the results obtained in Exercise 4.3.

A Material for Refreshing Basic Knowledge

MATLAB pills: If you want to use MATLAB and are not so familiar with it, you can find it useful to follow these video pills:

1. Introduction: https://youtu.be/J2-R_Hw4Ak8
2. Variables and Scripts: <https://youtu.be/oI4ZP-GxEuU>
3. Functions: https://youtu.be/C_59NdVeD0Q
4. Arrays and Matrices (part 1): <https://youtu.be/17TClaukIkE>
5. Indexing - Accessing Elements in Arrays and Matrices: <https://youtu.be/OB1I3FJn9Sg>
6. Operations with Arrays and Matrices: <https://youtu.be/J-gzenf-4bY>
7. Arrays and Matrices (part 2): <https://youtu.be/e9CdE95va-g>
8. Logical Indexing: https://youtu.be/_vjDMKF9zcQ

Background (Numerical Methods for Linear Algebra): if you need to consolidate your background in basic numerical methods for linear algebra, you may find it helpful to watch the following video lectures: from Portale della Didattica (general part, not the course page!) follow **Materiale** -> **Lezioni on-line** -> **Primo Anno** -> **Linear Algebra and Geometry**.

The suggested lectures are those delivered by Prof. Dabbene, and should be enough to watch lectures 5, 9, 13 (neglect the part on polynomials), 22, 29, and 33. Italian students may prefer to watch the lectures in Italian, then just look for Algebra Lineare e Geometria and look at lectures from Prof.ssa Scuderi (lectures 4, 8, 12, 24 [ignore the initial part on approximation], 28, 30).

B Test Functions

When you have to test an optimization method, it is useful to use some “standard” test functions². Here we present some test functions that will be used in the laboratories.

B.1 The Rosenbrock and the Himmelblau Test Functions

In this section we introduce the the 2-dimensional *Rosenbrock* function and the *Himmelblau* function that are the functions f_2 and f_3 , respectively, in the file `test_functions2.mat`. The other function, denoted as f_1 , is instead a simple paraboloid.

- **Paraboloid** ($n = 2$):

$$\begin{aligned} f_1(x_1, x_2) &= x_1^2 + 4x_2^2 + 5 \\ \nabla f_1(x_1, x_2) &= \begin{bmatrix} 2x_1 \\ 8x_2 \end{bmatrix} \\ H_{f_1}(x_1, x_2) &= \begin{bmatrix} 2 & 0 \\ 0 & 8 \end{bmatrix} \quad (constant) \\ \text{global minima} &= \{f_1(0, 0) = 5\} \\ \text{test } \mathbf{x}^{(0)} &= \{(0, 0), (5, 0)\} \end{aligned} \tag{31}$$

²E.g., see https://en.wikipedia.org/wiki/Test_functions_for_optimization

- **Rosenbrock** ($n = 2$):

$$\begin{aligned}
f_2(x_1, x_2) &= 100(x_2 - x_1^2)^2 + (1 - x_1)^2 \\
\nabla f_2(x_1, x_2) &= \begin{bmatrix} 400x_1^3 - 400x_1x_2 + 2x_1 - 2 \\ 200(x_2 - x_1^2) \end{bmatrix} \\
H_{f_2}(x_1, x_2) &= \begin{bmatrix} 1200x_1^2 - 400x_2 + 2 & -400x_1 \\ -400x_1 & 200 \end{bmatrix} \\
\text{global minima} &= \{f_2(1, 1) = 0\} \\
\text{test } \mathbf{x}^{(0)} &= \{(1.2, 1.2), (-1.2, 1)\}
\end{aligned} \tag{32}$$

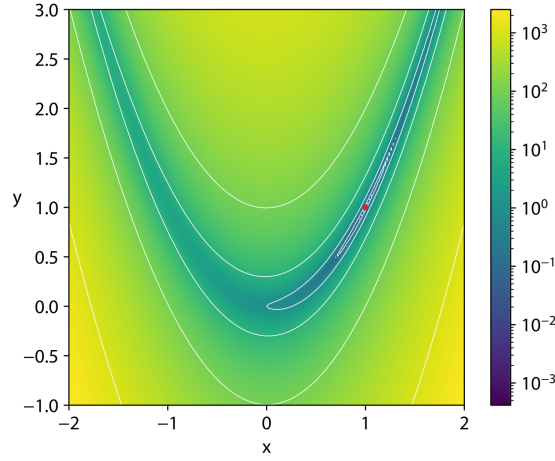


Figure 5: Rosenbrock function ($n = 2$), top view

- **Himmelblau**:

$$\begin{aligned}
f_3(x_1, x_2) &= (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2 \\
\nabla f_3(x_1, x_2) &= \begin{bmatrix} 4x_1^3 + 4x_1x_2 - 42x_1 + 2x_2^2 - 14 \\ 4x_2^3 + 4x_1x_2 - 26x_2 + 2x_1^2 - 22 \end{bmatrix} \\
H_{f_3}(x_1, x_2) &= \begin{bmatrix} 12x_1^2 + 4x_2 - 42 & 4x_1 + 4x_2 \\ 4x_1 + 4x_2 & 12x_2^2 + 4x_1 - 26 \end{bmatrix} \\
\text{global minima} &= \left\{ f_3(\mathbf{x}^{*(i)}) = 0 \mid i = 1, \dots, 4 \right\}, \quad \text{where} \\
\mathbf{x}^{*(1)} &= (3, 2), \quad \mathbf{x}^{*(2)} \simeq (-2.805118, 3.131312) \\
\mathbf{x}^{*(3)} &\simeq (-3.779310, -3.283186), \quad \mathbf{x}^{*(4)} \simeq (3.584428, -1.848126) \\
\text{test } \mathbf{x}^{(0)} &= \{(0, 0), \text{random in } [-4, 4]^2\}
\end{aligned} \tag{33}$$

B.2 n -dimensional Test Function

In this section, we introduce the n -dimensional, parametric, *Rosenbrock* function. See the 2-dimensional version in Section B.1.

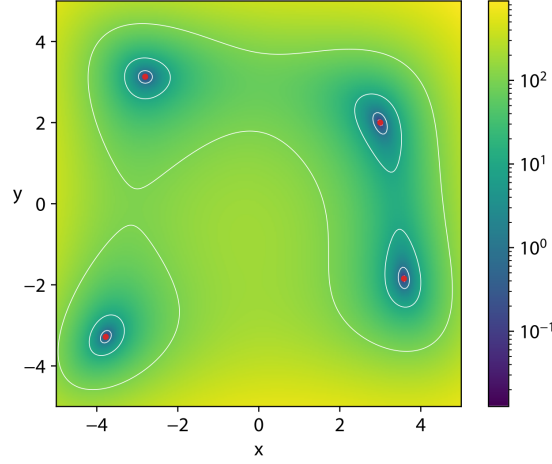


Figure 6: Himmelblau function, top view

- **Parametric Rosenbrock** ($\alpha > 0$, $n \in \mathbb{N}$):

$$\begin{aligned}
 f(\mathbf{x}) &= \sum_{i=1}^{n-1} (\alpha(x_{i+1} - x_i^2)^2 + (1 - x_i)^2) \\
 \nabla f(\mathbf{x}) &= \dots \text{exercise} \dots \\
 Hf(\mathbf{x}) &= \dots \text{exercise} \dots \\
 \text{global minima} &= \{f(1, \dots, 1) = 0\} \\
 \text{test } \mathbf{x}^{(0)} &= \{(1.2, \dots, 1.2), (-1.2, 1, \dots, -1.2, 1)\} \quad (\text{assuming } n \text{ even})
 \end{aligned} \tag{34}$$

Hints for computing derivatives:

$$\frac{\partial}{\partial x_i} f(\mathbf{x}) = \frac{\partial}{\partial x_i} (\alpha(x_{i+1} - x_i^2)^2 + (1 - x_i)^2 + \alpha(x_i - x_{i-1}^2)^2 + (1 - x_{i-1})^2), \tag{35}$$

for each $i = 2, \dots, n-1$; for $i = 1, n$, small changes occur:

$$\begin{aligned}
 \frac{\partial}{\partial x_1} f(\mathbf{x}) &= \frac{\partial}{\partial x_1} (\alpha(x_2 - x_1^2)^2 + (1 - x_1)^2) \\
 \frac{\partial}{\partial x_n} f(\mathbf{x}) &= \frac{\partial}{\partial x_n} (\alpha(x_n - x_{n-1}^2)^2 + (1 - x_{n-1})^2),
 \end{aligned} \tag{36}$$

Moreover, you can generalize (35) to the case of second order derivatives for computing the Hessian, observing also that most of its elements are zeros; specifically, the Hessian is tridiagonal and symmetric (see Figure 7):

$$\text{if } j < (i-1) \text{ or } j > (i+1) \quad \Rightarrow \quad \frac{\partial^2}{\partial x_i \partial x_j} f(\mathbf{x}) = 0. \tag{37}$$

Both the gradient and the Hessian of the function can be written directly as a function handle; nonetheless, if you prefer to have a more readable code, you can write them as a Matlab function instead (loading them as function handle variables just for running the optimization functions).

Hints for computing Finite Differences:

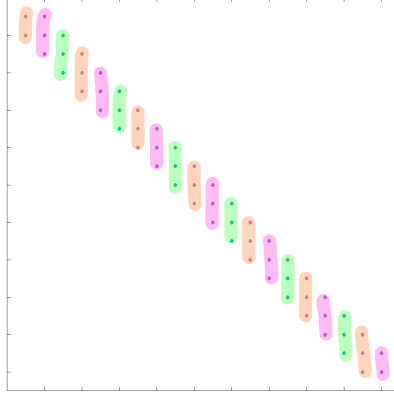


Figure 7: Scheme of the Hessian for the n -dimensional parametric Rosenbrock function. Colors show an example of exploiting the sparsity of the Hessian if approximated as the Jacobian of the exact gradient.

Let $f_1 : \mathbb{R}^2 \rightarrow \mathbb{R}$ and $f_2 : \mathbb{R} \rightarrow \mathbb{R}$ be such that $f_1(y_1, y_2) = \alpha(y_1 - y_2^2)^2$ and $f_2(y) = (1 - y)^2$. Therefore, we have that

$$f(\mathbf{x}) = \sum_{i=1}^{n-1} (\alpha(x_{i+1} - x_i^2)^2 + (1 - x_i)^2) = \sum_{i=1}^{n-1} (f_1(x_{i+1}, x_i) + f_2(x_i)) \quad (38)$$

and

$$\begin{aligned} f(\hat{\mathbf{x}} + h\mathbf{e}_i) &= (f_1(\hat{x}_2, \hat{x}_1)) + f_2(\hat{x}_1)) + \dots + \\ &\quad + (f_1(\hat{x}_i + h, \hat{x}_{i-1}) + f_2(\hat{x}_{i-1})) + (f_1(\hat{x}_{i+1}, \hat{x}_i + h) + f_2(\hat{x}_i + h)) + \dots + \\ &\quad + (f_1(\hat{x}_n, \hat{x}_{n-1}) + f_2(\hat{x}_{n-1})), \quad \forall i = 2, \dots, n-1, \\ f(\hat{\mathbf{x}} + h\mathbf{e}_1) &= (f_1(\hat{x}_2, \hat{x}_1 + h)) + f_2(\hat{x}_1 + h)) + \dots + \\ &\quad + (f_1(\hat{x}_n, \hat{x}_{n-1}) + f_2(\hat{x}_{n-1})), \\ f(\hat{\mathbf{x}} + h\mathbf{e}_n) &= (f_1(\hat{x}_2, \hat{x}_1)) + f_2(\hat{x}_1)) + \dots + \\ &\quad + (f_1(\hat{x}_n + h, \hat{x}_{n-1}) + f_2(\hat{x}_{n-1})). \end{aligned} \quad (39)$$

Then, for example:

$$\begin{aligned} f(\hat{\mathbf{x}} + h\mathbf{e}_i) - f(\hat{\mathbf{x}}) &= f_1(\hat{x}_i + h, \hat{x}_{i-1}) + (f_1(\hat{x}_{i+1}, \hat{x}_i + h) + f_2(\hat{x}_i + h)) - \\ &\quad - (f_1(\hat{x}_i, \hat{x}_{i-1}) + (f_1(\hat{x}_{i+1}, \hat{x}_i) + f_2(\hat{x}_i))), \quad \forall i = 2, \dots, n-1, \\ f(\hat{\mathbf{x}} + h\mathbf{e}_1) - f(\hat{\mathbf{x}}) &= (f_1(\hat{x}_2, \hat{x}_1 + h)) + f_2(\hat{x}_1 + h) - (f_1(\hat{x}_2, \hat{x}_1) + f_2(\hat{x}_1)), \\ f(\hat{\mathbf{x}} + h\mathbf{e}_n) - f(\hat{\mathbf{x}}) &= f_1(\hat{x}_n + h, \hat{x}_{n-1}) - f_1(\hat{x}_n, \hat{x}_{n-1}). \end{aligned} \quad (40)$$

N.B.: all these observations can be generalized to the diagonals of the Hessian matrix (see Figure 7) if computed with second order FD (not as approximated Jacobian of the exact gradient).

References

- [1] Jorge Nocedal and Stephen J. Wright. *Numerical optimization*. Second. 9781447122234. Springer, 2012. ISBN: 9780387303031. DOI: [10.1007/978-1-4471-2224-1_2](https://doi.org/10.1007/978-1-4471-2224-1_2).
- [2] Sandra Pieraccini. “Numerical Optimization for Large Scale Problems and Stochastic Optimization”. In: *Draft slides* (2023).